



SolGen: Secure Smart Contract Code Generation Using Large Language Models Via Masked Prompting

Md Tauseef Alam
Indian Institute of Technology
Patna
Patna, India
tauseef_2121cs04@iitp.ac.in

Sorbajit Goswami
Indian Institute of Technology
Patna
Patna, India
sorbajit_24ps57@iitp.ac.in

Khushi Singh
Indian Institute of Technology
Patna
Patna, India
khushi_2101cs37@iitp.ac.in

Raju Halder
Indian Institute of Technology
Patna
Patna, India
halder@iitp.ac.in

Abyayananda Maiti
Indian Institute of Technology
Patna
Patna, India
abyaym@iitp.ac.in

Soumyadip Banerjee
ACM member
Goa, India
soumyadip.bandyopadhyay@gmail.com

Abstract

Owing to the swift advancement of technology and the unfamiliarity of the execution environment, the development of Solidity smart contracts from scratch often results in significant vulnerabilities. In contrast, automated code generation enhances productivity, minimizes development time, and enables developers to focus on high-level tasks and fundamental logic. In consideration of these two viewpoints, this paper examines the utilization of large language models (LLMs) for the automatic generation of Solidity smart contracts based on specified criteria, while simultaneously ensuring the elimination of vulnerabilities through a novel masking strategy. To achieve this, we propose *SolGen*, a framework for generating secure Solidity smart contract code using LLMs. We assess the performance of existing LLMs (i.e. ChatGPT and Meta AI) for secure Solidity code generation. Our research indicates that ChatGPT outperforms Meta AI in performance, yielding a greater percentage of syntactically accurate and secure code. Additionally, we examine the impact of temperature adjustment on the security of generated contracts using an open-source LLM, Llama3. Our findings suggest that a temperature setting of 0.7 is optimal for the generation of Solidity code, considerably exceeding the performance of both lower and higher settings (0.1 and 1.2), especially with regard to the compilability of the code.

Keywords

Smart Contracts, Code Generation, Large Language Models, Prompt Engineering

ACM Reference Format:

Md Tauseef Alam, Sorbajit Goswami, Khushi Singh, Raju Halder, Abyayananda Maiti, and Soumyadip Banerjee. 2025. SolGen: Secure Smart Contract Code Generation Using Large Language Models Via Masked Prompting. In *18th Innovations in Software Engineering*



This work is licensed under a Creative Commons Attribution International 4.0 License.

ISEC 2025, Kurukshetra, India

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1424-5/25/02

<https://doi.org/10.1145/3717383.3717394>

Conference (ISEC 2025), February 20–22, 2025, Kurukshetra, India. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3717383.3717394>

1 Introduction

Automatic code generation refers to the automatic transformation of a specification or a requirement into source code as well as an executable application. This strategy advances software engineering by enhancing productivity, reducing software development time, and allowing developers to focus on high-level tasks and core logic. Automating the code generation process enables developers to optimize their work processes and prevent potential problems in the processes of software development. Nowadays, Ethereum platform is most popular blockchain platform on which smart contracts are deployed. These smart contracts written in programming languages like Solidity, are increasingly gaining popularity in various sectors such as healthcare, education, gaming, finance, and cyber-physical systems [13]. The use and popularity of Solidity-based smart contracts, especially those that deal with financial operations, have placed them on the radar of attackers who are significantly motivated by the finances due to the nature of these contracts being open-sourced. Some of the vulnerabilities include reentrancy, integer overflow/underflow, transaction ordering dependency, denial of service, and block-timestamp dependency [4]. It is highly important to ensure the safety of Solidity smart contracts before they are deployed due to the economic damages that a smart contract with effective vulnerability can inflict.

In last few years, the recent advancements with the evolution of transformers and large language models (LLMs) in the field of artificial intelligence have depicted these models' effectiveness in various Natural Language Processing (NLP) tasks like language translation, question answering, summarization, text generation, and grammar checking [3]. Among these models, OpenAI's GPT-4o [2] and Meta AI's Llama 3.1 [27] that are trained on diverse datasets, perform reasonably good across various application domains. Furthermore, the performance of these LLMs on code-related tasks makes them highly relevant for increasing productivity in a variety of fields, such as the generation of Solidity smart contracts. Therefore, the primary question that we are investigating in

this work is as follows: *Is it possible for LLMs to write smart contract codes from user input prompts, and to what extent are the contracts that are resulting from this are secure?*

This paper presents the results of substantial initial research that we conducted in order to analyze deficiencies in the code that was generated for smart contracts and to investigate the effectiveness of prompt engineering techniques in addressing these deficiencies. This paper also investigate the effective integration of large language models (LLMs) with formal method based tools to automatically generate secure, non-vulnerable Solidity smart contract code based on given specifications. Additionally, to guarantee that vulnerabilities are not present in the code, an innovative masking technique is applied. Therefore, in order to enhance the effectiveness of large language models (LLMs), a comprehensive understanding of the process of detecting defects and performing prompt engineering is absolutely necessary.

Moreover, the prompt engineering is both cost-effective as well as efficient in terms of improving LLM performance when compared to retraining, which requires a significant amount of time and resources. Considering this particular context, our research addresses the following key research questions (RQs):

RQ1: *Is the Solidity code compilable when generated by the closed source LLMs (i.e. Meta AI and ChatGPT)?*

RQ2: *How multi-round prompting technique effectively resolve the compilation issues of Solidity smart contracts generated by closed-source LLMs (such as Meta AI and ChatGPT)?*

RQ3: *Is the Solidity smart contract source code secure and non-vulnerable when generated by the closed source LLMs (i.e. Meta AI and ChatGPT)?*

RQ4: *How masked prompting technique assists in addressing vulnerabilities within Solidity smart contracts code generated by closed-source LLMs (such as Meta AI and ChatGPT)?*

RQ5: *What effect does adjusting the temperature parameter of open source LLM have on the generation of secure and non-vulnerable Solidity smart contracts?*

2 Motivation

There are various advantages of AI-driven smart contract code generation using LLMs in terms of productivity and automation but at the same time it is very crucial to access and understand the quality of code. In recent times, the code generation by LLMs faces several challenges, such as ensuring the functional correctness, security, and complexity of the output [15]. Moreover, the quality of the generated code may be affected because the LLMs like ChatGPT and Meta AI are trained on publicly available datasets on internet which may compromise with the quality of the training data [5]. Thus we cannot guarantee and rely on the accuracy and security of such generated Solidity code.

The reentrancy vulnerability is one of the mischievous and notorious vulnerability in Solidity smart contracts which had lead to million of dollar losses up till now [6]. This attack allows the repeated re-execution of a function before all statements from its initial execution are fully completed, thereby exploiting the contract’s logic and leading to potentially significant financial losses. Likewise, vulnerabilities

in Solidity smart contracts such as integer overflow and underflow, transaction ordering dependency, denial of service, block-timestamp dependency, access control, etc., can result in significant financial losses [4].

An in depth and clear analysis of non-vulnerable Solidity code generation through LLM is necessary to tackle these issues. There are few preliminary studies [11, 23, 25] on generating smart contract code but none of the works in the literature have focused on generating secure and non-vulnerable code with the use of large language models and formal method tools. To bridge this gap, this paper primarily focus on syntactical correctness and security of the Solidity smart contract produced by LLMs by systematically evaluating it. In this paper, we target the top 10 vulnerabilities in Solidity smart contract as identified by the “OWASP Smart Contract Top 10”¹. Due to the popularity and widespread acceptance of ChatGPT and Meta AI [19], we use these two LLMs for our evaluations. Moreover, we explore the ability of these closed source user interactive LLMs to perform multi-round prompting and masked prompting to iteratively fix defective code containing errors. We aim to highlight limitations and challenges in generating Solidity code by using LLMs, and focusing mainly on correctness and security of the code. Our goal is to give information that can help making AI-driven Solidity smart contract development tools better in the future.

3 Contributions

To summarize, the key contributions of this paper are:

- We conduct preliminary investigation for the feasibility and quality of LLM-based Solidity smart contract code generation.
- We compare and evaluate two well-known LLMs (i.e., ChatGPT and Meta AI) while generating Solidity code in terms of syntactic correctness and security. The Solidity code generated through LLMs might have problems and issues in both of these areas as observed by the results.
- We propose **SolGen**, a framework for generating Solidity source code for the Ethereum blockchain. **SolGen** uses closed-source LLMs (e.g. ChatGPT and Meta AI) and static analysis to generate secure Solidity smart contracts through multi-round and masked prompting.
- We show how modifying the temperature parameter produces secure Solidity smart contracts.
- Our research advances the knowledge of how LLMs can enhance best practices for Ethereum smart contract development, specifically in terms of Solidity code generation.

All experimental artifacts have been made available on GitHub repository : <https://github.com/mdtauseefalam/SolGen>.

4 Related Work

Karanjai et al. [11] had explored and examined potential of Google PaLM2 and ChatGPT for smart contract generation highlighting the need for analysis of input quality and

¹<https://owasp.org/www-project-smart-contract-top-10/>

diversity to access its strengths and limitations. Osaе et al. [23] introduced MazzumaGPT, a fine-tuned large language model customized for generating smart contract code. When compared to ChatGPT, it showed improved performance in generating compilable code but was unable to generate non-vulnerable code. The authors in [25] investigated the impact of fine-tuned GPT-J in synthesizing non-vulnerable smart contracts. Their primary focus is on incremental fine-tuning of the GPT-J to auto-complete the codes, while restricting the generation of vulnerable smart contract codes.

In [14] Li et al. fine-tuned the LLM for secure C/C++ code generation. They used GPT-J and showed that fine-tuning improved the percentage of generating non-vulnerable code to 70.4% for C and 64.5% for C++, which is a 10% improvement for C and a slight increase for C++ when compared to the pre-trained model. In a similar direction, Liu et al. [16] evaluated the quality of generating C, C++, and Java code by using ChatGPT. To generate functionally correct and secure code they proposed a multi-round fixing process, thus highlighting the limitations and issues of ChatGPT. In a related study, H. Pearce et al. [24] explored Copilot’s performance across three distinct code generation parameters for C, Python and Verilog: diversity of weaknesses, diversity of prompts, and diversity of domains. They found that 40% of the generated codes was vulnerable. In [10] Han et al. proposed a framework, ARCHCODE, that leverages in-context learning to extract and organize requirements from textual descriptions and infer unexpressed requirements. ARCHCODE then generates Python and Java code snippets and test cases based on these inferred requirements. Also, the authors have explored the performance of ChatGPT and other models for code generation and vulnerability detection. Guo et al. [9] introduced a technique CodeFast, to accelerate inference for code generation in Python, Javascript, Go and C++ languages. CodeFast performance is increased by identifying unnecessary excess tokens early in the generation process and terminates inference. Similarly, in the work [12], Koziolęk et al. proposed an automated approach using LLMs for generating test cases for Programmable Logic Controllers (PLCs). They generate inputs and expected outputs for IEC 61113-3 function blocks. Their method shows the application of LLMs in synthesizing test cases from code prompts.

5 Proposed Approach

This work includes the use of instruction-tuning [30], iterative prompt refinement through feedback [18], and masked prompting [29] techniques for the generation of secure Solidity smart contract code with the absence of vulnerabilities. Figure 1 gives an overview of our proposed methodology. The main idea of this proposal is to generate a compilable, secure and non-vulnerable Solidity source code. This approach also takes into consideration the token limit of ChatGPT and MetaAI LLMs for generating the secure source code. Our proposed methodology is divided into two phases: (a) Phase 1: Generating the compilable source code, and (b) Phase 2: Generating the non-vulnerable secure Solidity code. We now describe below each of these two phases in detail. Table

1 outlines the notations that are utilized in our proposed methodology and their respective descriptions.

Table 1: Notations used in our methodology.

Notations	Descriptions
M	These are the LLMs used for source code generation (i.e. ChatGPT, Meta AI).
Pr	It is the prompt supplied to M .
C_i	It is the Solidity source code generated by LLM M at the i th iteration.
$Err(C_i)$	It is the set of compilation errors detected in the generated Solidity code C_i .
$Vul(C_i)$	It is the set of vulnerabilities that are identified in the generated code C_i .
R	This is the feedback for compilation errors identified by the Remix IDE.
$\langle mask \rangle$	This is for representing a placeholder for detected vulnerable code segments.

5.1 Phase 1: Generating the Compilable Solidity Source Code

The first phase involves iteratively refining the prompt Pr based on the feedback from the Remix IDE² until the generated source code C_i is compilable successfully.

(a) *Prompt construction*: To this aim, ① we initially select 20 unique and different Smart Contract Requirement Specification (SCRS) and then understand its functionality in detail. After this, ② we move to design the practical prompt as an input to the LLMs. Instead of merely optimizing the prompts for maximizing the LLM’s performance execution, our approach here focuses on creating a prompt that reflects the real-world smart contract code generation scenarios. In drafting the prompt template, we took the reference online resources for prompt design (e.g., OpenAI Cookbook [22] and PromptBase [1]) aiming to align with smart contract generation tasks. Thus, we develop the guiding principle for our prompt: “providing LLMs with sufficient relevant information and at the same time using their dialogue capabilities.” Figure 2 represents a sample prompt constructed and given as an input to the LLMs for generating Solidity smart contract code.

(b) *Initial Submission of the Prompt and Generating the Code*: When the appropriate prompts for the given smart contract specifications are created, then they are ③ fed as input to the LLMs (i.e., ChatGPT and MetaAI). In step ④, the LLM under consideration generates a Solidity code as an output to the prompt, after incorporating the feedback from errors identified in the previous iteration (with no error feedback context provided in the initial round). We then take into consideration only the code snippet generated by ChatGPT and Meta AI, isolating the content present between two triple backticks, while discarding the text.

$$C_1 = M(Pr)$$

here C_1 is representing the Solidity code generated in the initial iteration.

(c) *Identifying the Error and Including the Feedback*: The generated Solidity code may not successfully compile. Thus, ⑤ it is checked for compilability using the compilers in the Remix IDE. Now, ⑥ if the code is not compilable and ⑦ contain errors, ⑧ we incorporate the feedback informations from Remix IDE to refine the prompt and feeding

²<https://remix.ethereum.org/>

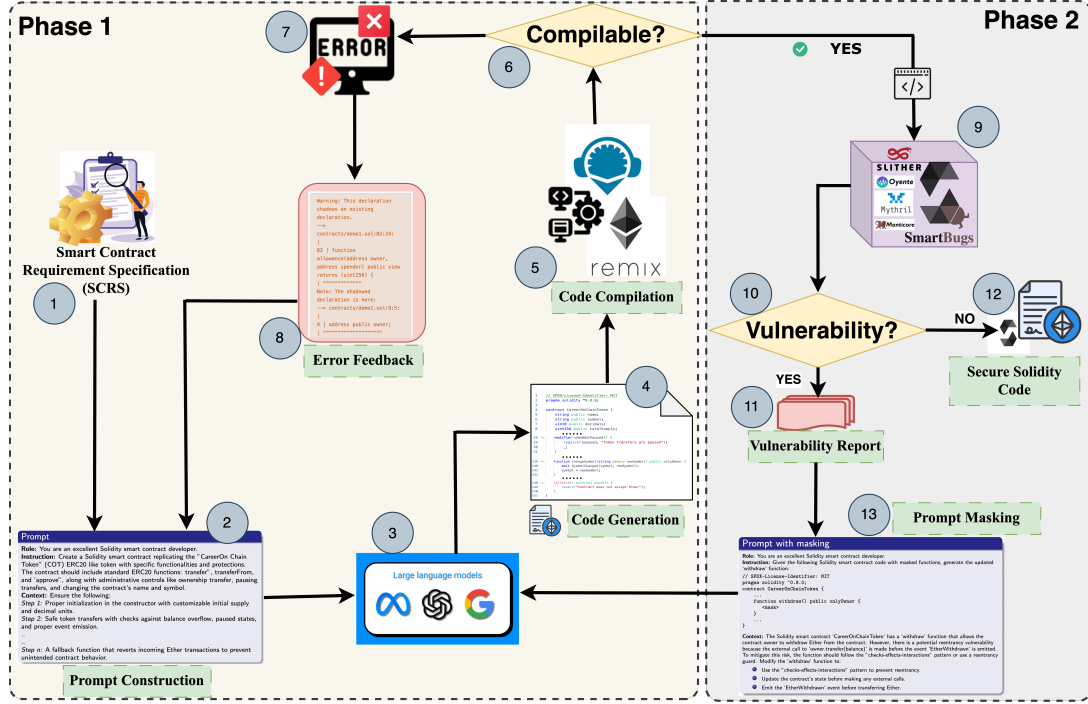


Figure 1: The workflow of our proposed SolGen framework.

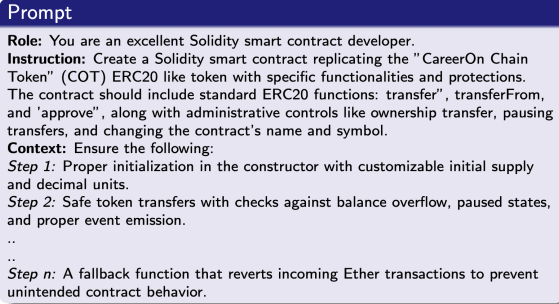


Figure 2: An instance of prompt construction.

it back to the LLMs for generating a new round of Solidity code. This iterative process continues until the LLM under consideration generates a new version of the code snippet when compared to the previous round and all errors in the generated smart contract code are resolved and fixed.

In short after generating C_1 , we check for compilation errors using Remix IDE as:

$$Err(C_1) = RemixIDE(C_1)$$

If $Err(C_1) \neq \emptyset$ (i.e., there is presence of errors in the code), we extract and identify the feedback report R from Remix IDE and then refine the prompt:

$$Pr_{i+1} = Pr_i + R$$

This process continues until we get:

$$Err(C_i) = \emptyset$$

which means that the generated code is successfully compilable.

Once a compilable Solidity smart contract code is produced, we proceed to the next phase (i.e., Phase 2) of checking the source code for the presence of potential vulnerabilities.

5.2 Phase 2: Generating the non-vulnerable secure Solidity code

Once a compilable code C_i is generated, we proceed with the second phase. Phase 2 involves iteratively removing vulnerabilities from the given code after its detection.

(a) *Vulnerability Detection/Identification*: To detect vulnerabilities, (9) we make use of the current state-of-the-art (SOTA) formal verification tools available publicly to detect vulnerabilities within the given Solidity source code. (10) These tools generate a vulnerability report that provides the detail of the vulnerability present. (12) If no vulnerabilities are identified by the tools, we terminate the process. Otherwise, (11) if the generated source code contains vulnerabilities, the affected functions/methods of the contract code are identified and are highlighted in the generated vulnerability report.

For vulnerability detection in Solidity code, several formal method based tools have been developed by research community and industry which are publicly available. Out of these, the most popular and widely used tools include Slither [7], Manticore [20], Mythril [21], SmartCheck [26], Securify [28], and Oyente [17]. The docker image of each of these tools is integrated and is available in the SmartBugs tool [8]. Among them, Slither and Mythril are the most actively maintained tools, with the recent GitHub commits

that are recorded in August 2024. Notably, Slither is the most popular tool within the developer community when compared with other SOTA formal method based tools, with a high rating of 5.3K stars (as of September 25, 2024) making Slither perfectly suitable for our study.

Thus, we use Slither tool for detecting vulnerabilities as described below:

$$Vul(C_i) = Slither(C_i)$$

If $Vul(C_i) = \emptyset$ (i.e., there are no vulnerabilities detected in code and the code is secure), the process terminates. Otherwise, feedback vulnerability report $F(Vul(C_i))$ is analysed to identify vulnerable functions or methods of the smart contract code.

(b) *Masking of the Vulnerable Smart Contract Code:* ⁽¹³⁾ A new prompt is constructed by masking the identified vulnerable functions or methods in the smart contract. This new prompt includes the vulnerable source code snippet that was generated by the LLM, which is masked with a special token called `<mask>`. Additionally, it includes the steps and useful information that are necessary to eliminate the identified vulnerability. Figure 3 demonstrates an example of a new constructed prompt with masking. This prompt is used to remove the reentrancy-vulnerable function ‘`withdraw()`’.

If $Vul(C_i) \neq \emptyset$ (i.e., vulnerability is detected in the code), the vulnerable chunk of the Solidity code is masked as:

$$C_{mask} = C_i - Vul(C_i) + \langle mask \rangle$$

Now, a new prompt Pr_{mask} is created with the instructions to fix the vulnerability and masked Solidity code as:

$$Pr_{mask} = C_{mask} + InstructionstoFix$$

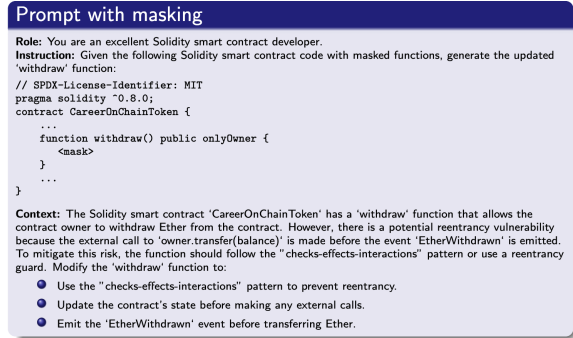


Figure 3: An instance of prompt with masking.

(c) *New Smart Contract Code Generation and Multiple Iteration:* The new modified prompt obtained from previous step is fed as input to the LLM for generating the Solidity code. The code for the smart contract that is produced is re-examined using Slither to determine whether or not it contains any vulnerabilities. If the vulnerability is detected then we repeat all the steps for generating non-vulnerable code. The repetition continues until a non-vulnerable version of Solidity code is obtained. The LLM generates a new version of the Solidity source code using the modified prompt as:

$$C_{i+1} = M(Pr_{mask})$$

This process repeats again and again until:

$$Vul(C_{i+k}) = \emptyset$$

which means that a non-vulnerable, secure Solidity code is produced.

The complete methodology for generating a secure, non-vulnerable Solidity smart contract is being outlined in the Algorithm 1. This methodology starts at line number 1 with the construction and designing of the initial prompt. In line number 2 the prompt is then fed to the large language model (LLM) 'M' as an input to generate the first iteration of the Solidity code as output, denoted as C_1 . After that, we use compilers in Remix IDE to check for compilation errors which is represented as $Err(C_i)$. In line number 3, the algorithm loops as long as the error $Err(C_i)$ is not null (i.e., error present) and the newly generated code C_i is not identical (i.e not same) to the code from the previous iteration C_{i-1} (except for the first iteration). In each iteration the error details R from Remix IDE is used for refining the prompt as shown in line number 4. Also at line 5 the new code C_{i+1} is generated using refined prompt Pr_{i+1} and feeding it to 'M'.

At line number 7 the code is checked for compilation, if it is successfully compilable (i.e., $Err(C_i) = \emptyset$), we then check for the presence of vulnerabilities using Slither tool. The vulnerability report represented as $Vul(C_i)$ is analyzed. Our algorithm terminates if no vulnerability (i.e., $Vul(C_i) = \emptyset$) is detected.

However, when the vulnerabilities are detected ($Vul(C_i) \neq \emptyset$), the loop continues from line 11 to line 13 iteratively. In line number 11, masking of the vulnerable code is done by replacing the vulnerable function using `<mask>` to produce masked code C_{mask} . The C_{mask} is then concatenated with the steps to fix the vulnerabilities on line 12 to generate a new prompt Pr_{mask} . After this the iterative regeneration of smart contract code is done until it becomes non-vulnerable by feeding the masked prompt to LLM M. Thus we observe that the multiple round refinement procedure ensures that the generated code is syntactically correct and is also secure against potential vulnerabilities.

Algorithm 1 Secure Non-Vulnerable Solidity Code Generation

- 1: Initialize Pr_1
 - 2: Generate $C_1 = M(Pr_1)$
 - 3: **while** $Err(C_i) \neq \emptyset$ **and** $C_i \neq C_{i-1}$ **do**
 - 4: Refine the prompt with feedback R
 - 5: Regenerate the code $C_{i+1} = M(Pr_{i+1})$
 - 6: **end while**
 - 7: **if** $Err(C_i) = \emptyset$ **then**
 - 8: Check for the vulnerabilities $Vul(C_i)$
 - 9: **end if**
 - 10: **while** $Vul(C_i) \neq \emptyset$ **do**
 - 11: Mask the vulnerable code $C_{mask} = C_i - Vul(C_i) + \langle mask \rangle$
 - 12: Generate masked prompt $Pr_{mask} = C_{mask} + InstructionstoFix$
 - 13: Regenerate the code $C_{i+1} = M(Pr_{mask})$
 - 14: **end while**
-

Selected LLMs. We select two state of the art popular large language models (LLMs) (a) ChatGPT and (b) Meta AI to test the effectiveness of our proposed approach. OpenAI and Meta developed and maintains ChatGPT and Meta AI respectively. Both models under consideration provides interactive user interfaces and are highly sophisticated as ChatGPT uses GPT-4o language model consisting of 1.76 trillion parameters whereas Meta AI uses Llama 3.1 model containing 405 billion parameters. These LLMs manage a diverse range of tasks such as code completion, summarization, and text generation. In this paper, we explore their potential for generating Solidity smart contracts.

Every LLM have a constraint of token limit which means the maximum input they can process along with maximum output they generate. Thus, to avoid information loss due to generated large smart contract code in combination with error details during re-prompting is very crucial. This bottleneck is handled by both GPT-4o and Llama 3.1 each having context window of 128k tokens. Thus, our methodology takes utmost care that there is no information loss during complete cycle of generating compilable, non-vulnerable and secure Solidity smart contract code.

6 Experimental Evaluation

6.1 Experimental Setup

We select two popular user interface based closed source LLMs (Meta AI and ChatGPT) to tackle the Research Questions outlined in Section 1. They are evaluated and compared for generating secure non vulnerable Solidity smart contracts. Furthermore, we use an open source LLM, Llama3, to investigate the impact of changing its temperature parameter. The experiments for Llama3 model were executed on a single-core server with 80 GB GPU (i.e. NVIDIA A100 80GB PCIe).

6.2 Comparison between Meta AI and ChatGPT

In this section, we address research questions RQ1 to RQ4 outlined in Section 1. We perform the experiments on 20 unique selected case studies that contain “OWASP Smart Contract Top 10” vulnerabilities. Tables 2 and 3 showcase the performance of Meta AI and ChatGPT in generating Solidity code based on various input prompts. It also highlight the multi round prompting technique used to generate compilable Solidity code. The ‘#RPC’ column in Tables 2 and 3 refers to the number of re-prompts required for generating compilable code. The ‘-’ and ‘*’ symbol in Tables 2 through 8 indicates that the LLM failed to generate valid Solidity code and produced either gibberish or irrelevant output.

RQ1: *Is the Solidity code compilable when generated by the closed source LLMs (i.e. Meta AI and ChatGPT)?*

Tables 2 and 3 shows that out of 20 different unique prompts, the code generated by 8 prompts using Meta AI is directly compilable. In contrast with Meta AI, ChatGPT performance is more superior because it produces compilable code for 13 out of 20 prompts. However, it is observed that LLMs do not consistently generate compilable code on the first attempt.

Table 2: Overview of Solidity Smart Contract Code Generation and Re-prompting using Meta AI (Llama 3.1)

Sn.	Prompts	Generated Code				
		What the code is about	Solc Version	#Function	#LOC	#RPC
1	P1	Solidity ERC20 token with pausing, ownership, and reentrancy protection.	0.8.0	12	175	6
2	P2	Solidity contract for document submission, signing, and secure Ether withdrawals.	0.8.0	7	98	8
3	P3*	Solidity token contract with transfers, approvals, burning, and secure Ether withdrawals.	-	-	-	7
4	P4	Solidity contract for file proofs, QR codes, and secure Ether withdrawals.	0.8.0	11	132	2
5	P5	Solidity contract for Ether and token transfers with reentrancy protection.	0.8.0	10	151	3
6	P6	Solidity digital notary with hash registration, ownership changes, and jackpot system.	0.8.0	23	281	10
7	P7	Solidity contract for event metadata, operator management, multihash handling, and proof hash.	0.8.0	31	302	4
8	P8*	ERC20 “SaveWon” token with safe arithmetic, minting, burning, and reentrancy protection.	-	-	-	5
9	P9*	ERC20 “Quras Token” with secure transfers, ownership management, and event emissions.	-	-	-	4
10	P10	EIP20 “HotDollarsToken” with token transfers, approvals, and balance checks.	0.8.0	8	64	0
11	P11	Solidity ERC20, Allowance and Transfer function. Check the amount of tokens that an owner owns.	0.8.0	7	66	0
12	P12	Solidity ERC20, Allowance and Transfer function. Check the amount of tokens that an owner has. Mapping function to track record of private balance.	0.8.0	11	56	0
13	P13	ERC20 Token which contains approve function and token purchase and sale function. Sale function allows users to sell tokens back to the contract in exchange for Ether.	0.8.0	9	66	0
14	P14	Stock market related solidity code which includes the function to approve company implemented in share market and to control internal company data and to set initial stock price.	0.8.0	10	79	0
15	P15	ERC20 Token which contains and token purchase and sale function. Increases the user’s token balance by the purchased amount.	0.8.0	11	71	1
16	P16	ERC20 Token which contains and token purchase and sale function. Increases the user’s token balance by the purchased amount.	0.8.0	10	78	0
17	P17	ERC20 Token which contains approve function, supply function, and mapping balance of function.	0.8.0	10	71	1
18	P18	ERC20 Token which contains token sale and token purchase function and supply function. Ensures the user has enough tokens to sell.	0.8.0	11	83	0
19	P19	ERC721 Token which contains allowance function. Increases the user’s token balance by the purchased amount. The number of tokens the user wants to sell.	0.8.0	13	90	1
20	P20	A basic banking smart contract with transfer and withdraw function. A balanceOf function to track record of the balance.	0.8.0	8	46	0

RQ2: *How multi-round prompting technique effectively resolve the compilation issues of Solidity smart contracts generated by closed-source LLMs (such as Meta AI and ChatGPT)?*

To assess the effectiveness of the LLM in generating compilable code, we employ a multi-round prompting approach through iteratively refining the prompts and then feeding it to the model until a compilable result is produced. This process is repeated until the LLM generates a new version of the code compared to the previous iteration. In Table 2, we see that for Meta AI, most prompts required fewer than 10 rounds of re-prompting to yield compilable code. However, Meta AI fails to generate compilable code for three specific prompts (P3, P8, and P9), becoming saturate after 7, 5, and 4 rounds, respectively.

In contrast, as shown in Table 3, ChatGPT requires fewer than 5 rounds of re-prompting for most prompts, with exceptions being P7 and P8. It is unable to generate compilable code for prompt P7, reaching saturation after 17 rounds. Along with this, we observe that Meta AI always use the same compiler version (i.e., ^0.8.0) across all prompts,

Table 3: Overview of Solidity Smart Contract Code Generation and Re-prompting using ChatGPT (GPT-4o)

Sn.	Prompts	Generated Code				
		What the code is about	Solc Version	#Function	#LOC	#RPC
1	P1	Solidity ERC20 token with pausing, ownership, and reentrancy protection.	0.8.0	18	151	1
2	P2	Solidity contract for document submission, signing, and secure Ether withdrawals.	0.8.19	10	87	3
3	P3	Solidity token contract with transfers, approvals, burning, and secure Ether withdrawals.	0.8.0	14	111	0
4	P4	Solidity contract for file proofs, QR codes, and secure Ether withdrawals.	0.8.19	10	119	0
5	P5	Solidity contract for Ether and token transfers with reentrancy protection.	0.8.19	9	85	0
6	P6	Solidity digital notary with hash registration, ownership changes, and jackpot system.	0.8.0	13	100	0
7	P7*	Solidity contract for event metadata, operator management, multihash handling, and proof hash.	-	-	-	17
8	P8	ERC20 "SaveWon" token with safe arithmetic, minting, burning, and reentrancy protection.	0.8.26	14	112	13
9	P9	ERC20 "Quras Token" with secure transfers, ownership management, and event emissions.	0.8.26	9	117	4
10	P10	EIP20 "HotDollarsToken" with token transfers, approvals, and balance checks.	0.8.0	8	88	0
11	P11	Solidity ERC20, Allowance and Transfer function. Check the amount of tokens that an owner owns.	0.8.0	11	91	1
12	P12	Solidity ERC20, Allowance and Transfer function. Check the amount of tokens that an owner has. Mapping function to track record of private balance.	0.8.0	4	47	0
13	P13	ERC20 Token which contains approve function and token purchase and sale function. Sale function allows users to sell tokens back to the contract in exchange for Ether.	0.8.0	10	94	1
14	P14	Stock market related solidity code which includes the function to approve company implemented in share market and to control internal company data and to set initial stock price.	0.8.0	9	85	0
15	P15	ERC20 Token which contains token purchase and sale function. Increases the user's token balance by the purchased amount.	0.8.0	9	78	0
16	P16	ERC20 Token which contains token purchase and sale function. Increases the user's token balance by the purchased amount.	0.8.0	7	78	0
17	P17	ERC20 Token which contains approve function, supply function, and mapping balance of function.	0.8.0	7	105	0
18	P18	ERC20 Token which contains token sale and token purchase function and supply function. Ensures the user has enough tokens to sell.	0.8.0	8	86	0
19	P19	ERC721 Token which contains allowance function. Increases the user's token balance by the purchased amount. The number of tokens the user wants to sell.	0.8.0	9	84	0
20	P20	A basic banking smart contract with transfer and withdraw function. A balanceOf function to track record of the balance.	0.8.0	4	42	0

whereas ChatGPT demonstrates a more dynamic approach, generating different compiler versions for different prompts.

These observations indicate that ChatGPT outperformed Meta AI in code generation following multi-round prompting, and demonstrating a higher tolerance for iterative feedback along with reduced susceptibility to early saturation. ChatGPT when compared with Meta AI, is more reliable for generating successful compilable Solidity smart contract code and depicts much better performance with multi-round prompting technique.

RQ3: *Is the Solidity smart contract source code secure and non-vulnerable when generated by the closed source LLMs (i.e. Meta AI and ChatGPT)?*

We make use of the popular static analysis tool Slither (v0.10.0), available with SmartBugs framework to identify vulnerabilities within smart contract code. Tables 4 and 5 provide a detailed overview of the detected vulnerabilities, including the number of vulnerable functions and the impact observed after applying masked prompting. We upload all the log files of the generated vulnerability reports to the GitHub repository mentioned in Section 3. Our analysis

reveals that all compilable code generated from both Meta AI and ChatGPT exhibits vulnerabilities. However, we focus on significant vulnerabilities, such as reentrancy, and disregard those with very low severity.

Table 4 shows that 10 out of 17 compilable Solidity codes have reentrancy vulnerabilities, generated by Meta AI. Similarly, Table 5 shows that 9 out of 19 compilable codes generated by ChatGPT contain the same vulnerability. In total, 83 vulnerable functions are identified in Meta AI's generated code, compared to 54 vulnerable functions in the code generated by ChatGPT.

From this analysis we conclude that neither of two LLMs generates fully secure and non-vulnerable Solidity smart contracts. However, ChatGPT produces fewer vulnerable Solidity codes compared to Meta AI, suggesting a marginally better security profile in the generated output.

RQ4: *How masked prompting technique assists in addressing vulnerabilities within Solidity smart contracts code generated by closed-source LLMs (such as Meta AI and ChatGPT)?*

We apply a masked prompting technique to address the issue of vulnerable functions generated by the LLMs. This approach involves iteratively masking a vulnerable function in the code until a new and unique version of code is generated when compared to the previous iteration. During this process, we observe the presence of some low-severity vulnerabilities that were outside the scope of our study and therefore we ignored it. These are labeled as "Yes but Not Fatal" in Tables 4 and 5.

As shown in Table 4, the Meta AI model does not perform well with the masking technique. It struggles to resolve reentrancy vulnerabilities, even after repeated masked re-prompting, as seen with compilable code samples C2, C4-C6, C14-C17, and C19-C20. In contrast, ChatGPT performs effectively with masked prompting as shown in Table 5. In most cases, ChatGPT is able to resolve reentrancy vulnerabilities after no more than four iterations. The only exception is compilable code C5, which fails to produce a secure, compilable code even after multiple rounds of masked re-prompting.

In conclusion, we claim that ChatGPT addresses reentrancy vulnerabilities more efficiently as compared to Meta AI. Notably, our observations indicate that the LLMs consistently generate smart contracts that are free from arithmetic vulnerabilities. However, both LLMs face challenges in producing a fully secure, non-vulnerable smart contract even after multiple masked re-prompts. Thus we suggest the research community to explore the workings of LLMs from within and further enhance the capabilities of LLMs for Solidity smart contract code generation.

6.3 Impact of Temperature Change

The temperature parameter in Large Language Models (LLMs) controls the randomness of the output by adjusting the probability distribution over possible tokens (words or characters) that the model can generate. In Tables 6, 7, and 8, the notation 'N/A' denotes "not applicable," indicating that the generated contracts which fail to compile are excluded from vulnerability assessment.

Table 4: Vulnerability Detection and Masking in Code Generated by Meta AI

Sn.	Compilable Code	Meta AI			
		Vulnerability	#Vul. Func.	#Masked Re-prompting	Vulnerability Exist?
1	C1	Yes but Not Fatal	4	1	Yes but Not Fatal
2	C2	Yes, Re-entrancy Vulnerability	5	2	Yes, Re-entrancy Vulnerability
3	C3*	-	-	-	-
4	C4	Yes, Re-entrancy Vulnerability	8	3	Yes, Re-entrancy Vulnerability
5	C5	Yes, Re-entrancy Vulnerability	3	2	Yes, Re-entrancy Vulnerability
6	C6	Yes, Re-entrancy Vulnerability	10	2	Yes, Re-entrancy Vulnerability
7	C7	Yes but Not Fatal	12	1	Yes but Not Fatal
8	C8*	-	-	-	-
9	C9*	-	-	-	-
10	C10	Yes but Not Fatal	10	3	Yes but Not Fatal
11	C11	Yes but Not Fatal	1	1	Yes but Not Fatal
12	C12	Yes but Not Fatal	0	3	Yes but Not Fatal
13	C13	Yes but Not Fatal	1	2	Yes but Not Fatal
14	C14	Yes, Re-entrancy Vulnerability	4	1	Yes, Re-entrancy Vulnerability
15	C15	Yes, Re-entrancy Vulnerability	6	3	Yes, Re-entrancy Vulnerability
16	C16	Yes, Re-entrancy Vulnerability	5	2	Yes, Re-entrancy Vulnerability
17	C17	Yes, Re-entrancy Vulnerability	1	3	Yes, Re-entrancy Vulnerability
18	C18	Yes but Not Fatal	4	1	Yes but Not Fatal
19	C19	Yes, Re-entrancy Vulnerability	6	2	Yes, Re-entrancy Vulnerability
20	C20	Yes, Re-entrancy Vulnerability	3	1	Yes, Re-entrancy Vulnerability

Table 5: Vulnerability Detection and Masking in Code Generated by ChatGPT

Sn.	Compilable Code	ChatGPT			
		Vulnerability	#Vul. Func.	#Masked Re-prompting	Vulnerability Exist?
1	C1	Yes, Re-entrancy Vulnerability	3	4	Yes but Not Fatal
2	C2	Yes, Re-entrancy Vulnerability	8	2	Yes but Not Fatal
3	C3	Yes but Not Fatal	6	2	Yes but Not Fatal
4	C4	Yes but Not Fatal	6	3	Yes but Not Fatal
5	C5	Yes, Re-entrancy Vulnerability	4	6	Yes, Re-entrancy Vulnerability
6	C6	Yes but Not Fatal	5	2	Yes but Not Fatal
7	C7*	-	-	-	-
8	C8	Yes but Not Fatal	3	1	Yes but Not Fatal
9	C9	Yes but Not Fatal	6	1	Yes but Not Fatal
10	C10	Yes but Not Fatal	2	1	Yes but Not Fatal
11	C11	Yes but Not Fatal	0	1	Yes but Not Fatal
12	C12	Yes but Not Fatal	3	1	Yes but Not Fatal
13	C13	Yes but Not Fatal	0	1	Yes but Not Fatal
14	C14	Yes, Re-entrancy Vulnerability	1	1	Yes but Not Fatal
15	C15	Yes, Re-entrancy Vulnerability	1	1	Yes but Not Fatal
16	C16	Yes, Re-entrancy Vulnerability	1	4	Yes but Not Fatal
17	C17	Yes but Not Fatal	0	1	Yes but Not Fatal
18	C18	Yes, Re-entrancy Vulnerability	1	3	Yes but Not Fatal
19	C19	Yes, Re-entrancy Vulnerability	1	3	Yes but Not Fatal
20	C20	Yes, Re-entrancy Vulnerability	3	2	Yes but Not Fatal

RQ5: What effect does adjusting the temperature parameter of open source LLM have on the generation of secure and non-vulnerable Solidity smart contracts?

We focused on open-source LLMs for our analysis because of the lack of customization options in closed source LLMs. We evaluate the performance of Meta’s state-of-the-art LLM, Llama3 model in generating smart contract code, by varying the temperature parameter under three distinct settings: low (0.1), medium (0.7), and high (1.2). We use 10 unique and distinct prompts to observe how temperature impacts the quality of the generated Solidity code. As shown in Tables 6, 7, and 8, we observe that at a temperature of 0.1, Llama3 produces 4 compilable codes out of 10 and none have

reentrancy vulnerabilities. For the medium temperature of 0.7, 6 out of 10 generated codes are compilable with no reentrancy vulnerabilities. However, when the temperature is increased to 1.2, only 3 compilable codes are produced and one of which contained reentrancy vulnerabilities.

Along with this, we also examined the compiler versions used across these temperature settings. At 0.1, the compiler version remained constant (0.8.0) across all prompts. At a temperature of 0.7, the compiler versions varied, ranging from 0.5.1 to 0.8.0. At 1.2, the compiler versions are the most dynamic that is from 0.4.21 to 0.8.10. This variability at the highest temperature setting reduces the accuracy and stability in Solidity smart contract code generation.

Table 6: Overview of Solidity Smart Contract Vulnerabilities Using the Llama3 Model with a 0.1 Temperature Setting

Sn.	Prompts	Llama3 Model (Temperature = 0.1)						
		What the code is about	Sole Version	#Func.	#LOC	Compilable?	Vulnerability?	# Vuln. Func.
1	P1	Solidity ERC20 token with pausing, ownership, and reentrancy protection.	0.8.0	8	91	No	N/A	N/A
2	P2	Solidity contract for document submission, signing, and secure Ether withdrawals.	0.8.0	18	135	No	N/A	N/A
3	P3	Solidity token contract with transfers, approvals, burning, and secure Ether withdrawals.	0.8.0	6	35	Yes	Yes but not fatal	4
4	P4	Solidity contract for file proofs, QR codes, and secure Ether withdrawals.	0.8.0	5	64	No	N/A	N/A
5	P5	Solidity contract for Ether and token transfers with reentrancy protection.	0.8.0	9	88	No	N/A	N/A
6	P6	Solidity digital notary with hash registration, ownership changes, and jackpot system.	0.8.0	6	41	Yes	Yes but not fatal	4
7	P7*	Solidity contract for event metadata, operator management, multihash handling, and proof hash.	-	-	-	-	-	-
8	P18	ERC20 Token which contains token sale and token purchase function and supply function. Ensures the user has enough tokens to sell.	0.8.0	3	63	Yes	Yes but not fatal	3
9	P19	ERC721 Token which contains allowance function. Increases the user's token balance by the purchased amount. The number of tokens the user wants to sell. Ensures the user has enough tokens to sell.	0.8.0	3	37	No	N/A	N/A
10	P20	A basic banking smart contract with transfer and withdraw function. A balanceOf function to track record of the balance.	0.8.0	3	21	Yes	Yes but not fatal	1

Table 7: Overview of Solidity Smart Contract Vulnerabilities Using the Llama3 Model with a 0.7 Temperature Setting

Sn.	Prompts	Llama3 Model (Temperature = 0.7)						
		What the code is about	Sole Version	#Func.	#LOC	Compilable?	Vulnerability?	# Vuln. Func.
1	P1	Solidity ERC20 token with pausing, ownership, and reentrancy protection.	0.7.0	5	89	Yes	Yes but not fatal	3
2	P2	Solidity contract for document submission, signing, and secure Ether withdrawals.	0.7.0	9	65	Yes	Yes but not fatal	8
3	P3	Solidity token contract with transfers, approvals, burning, and secure Ether withdrawals.	0.7.0	6	37	Yes	Yes but not fatal	5
4	P4	Solidity contract for file proofs, QR codes, and secure Ether withdrawals.	0.7.0	4	80	Yes	Yes but not fatal	2
5	P5	Solidity contract for Ether and token transfers with reentrancy protection.	0.8.0	11	104	No	N/A	N/A
6	P6	Solidity digital notary with hash registration, ownership changes, and jackpot system.	0.7.0	8	53	Yes	Yes but not fatal	6
7	P7	Solidity contract for event metadata, operator management, multihash handling, and proof hash.	0.5.1	13	94	No	N/A	N/A
8	P18*	ERC20 Token which contains token sale and token purchase function and supply function. Ensures the user has enough tokens to sell.	-	-	-	-	-	-
9	P19	ERC721 Token which contains allowance function. Increases the user's token balance by the purchased amount. The number of tokens the user wants to sell. Ensures the user has enough tokens to sell.	0.8.0	5	41	No	N/A	N/A
10	P20	A basic banking smart contract with transfer and withdraw function. A balanceOf function to track record of the balance.	0.8.0	4	29	Yes	Yes but not fatal	1

Table 8: Overview of Solidity Smart Contract Vulnerabilities Using the Llama3 Model with a 1.2 Temperature Setting

Sn.	Prompts	Llama3 Model (Temperature = 1.2)						
		What the code is about	Solc Version	#Func.	#LOC	Compilable?	Vulnerability?	# Vuln. Func.
1	P1	Solidity ERC20 token with pausing, ownership, and reentrancy protection.	0.8.0	8	127	No	N/A	N/A
2	P2	Solidity contract for document submission, signing, and secure Ether withdrawals.	0.8.4	10	96	No	N/A	N/A
3	P3	Solidity token contract with transfers, approvals, burning, and secure Ether withdrawals.	0.8.10	4	29	Yes	Yes but not fatal	3
4	P4	Solidity contract for file proofs, QR codes, and secure Ether withdrawals.	0.8.0	2	64	No	N/A	N/A
5	P5	Solidity contract for Ether and token transfers with reentrancy protection.	0.8.0	10	159	Yes	Yes, Re-entrancy Vulnerability	12
6	P6	Solidity digital notary with hash registration, ownership changes, and jackpot system.	0.8.0	20	144	No	N/A	N/A
7	P7	Solidity contract for event metadata, operator management, multihash handling, and proof hash.	0.5.1	11	104	No	N/A	N/A
8	P18	ERC20 Token which contains token sale and token purchase function and supply function. Ensures the user has enough tokens to sell.	0.4.21	5	45	No	N/A	N/A
9	P19	ERC721 Token which contains allowance function. Increases the user's token balance by the purchased amount. The number of tokens the user wants to sell. Ensures the user has enough tokens to sell.	0.8.0	9	116	No	N/A	N/A
10	P20	A basic banking smart contract with transfer and withdraw function. A balanceOf function to track record of the balance.	0.8.0	3	25	Yes	Yes but not fatal	1

Further, we analyze the number of vulnerable functions in the generated compilable code. At a temperature of 0.1, there are 12 vulnerable functions, which increases to 25 at 0.7, and then decreases to 16 at 1.2. Based on these findings, we conclude that higher temperatures (1.2) lead to more dynamic but less accurate results, producing fewer compilable codes with high risk of vulnerabilities. Between the low and medium settings, we find that a temperature of 0.7 is optimal for Solidity smart contract code generation, as it performs better than the lower temperature of 0.1 in compilability.

7 Conclusion

This paper highlights the limitations and evaluation of ChatGPT and Meta AI LLM while generating Solidity smart contract in terms of security and syntactic correctness. To address these shortcomings, we propose a framework *Sol-Gen*, that integrates closed-source LLMs with static analysis techniques to detect vulnerabilities and correct it through iterative multi-round and masked prompting.

The results of the experiment conducted by us revealed several key findings: (i) ChatGPT generates more number of syntactically correct Solidity source codes when compared to Meta AI with single-round prompting. Notably, we observe that ChatGPT also faces challenges in consistently producing successfully compilable Solidity code. (ii) After applying multi round prompting technique, we notice that ChatGPT is more reliable than Meta AI for generating compilable code. However, ChatGPT is unable to resolve

compilation error always. (iii) Both the LLMs are unable to produce fully secure and non-vulnerable Solidity codes. In contrast with Meta AI, ChatGPT produces comparatively less vulnerable codes implying a marginally better security profile. (iv) Both models generate Solidity smart contract which are free from arithmetic vulnerabilities. Furthermore, ChatGPT outperforms Meta AI in terms of avoiding reentrancy vulnerabilities.

Notably, we also witnessed that neither open source nor closed source LLMs were competent enough to independently generate secure, non-vulnerable smart contract code. Therefore, for generating secure smart contract code the human intervention is necessary and important. Along with this, our findings reveals the importance of adjusting temperature for improving security of generated contracts and thus contributing to the best practices for Ethereum smart contract development. Specifically, in terms of compilability we see that a temperature setting of 0.7 is optimal for Solidity code generation, as it significantly performs better than lower and higher temperature settings of 0.1 and 1.2.

8 Limitations

The scope of results and experiments performed in this work are limited by two factors: (1) ChatGPT and Meta AI are closed-source models, thus it restricts our ability to directly correlate the analysis results with the internal workings of these LLMs, which is a necessary step to fully understand the specific characteristics of the models. Moreover, the datasets used for training these LLMs are not disclosed

publicly which makes it difficult to determine the fact that whether the problems we tested were part of the training data used in the model. (2) Both ChatGPT and Meta AI are continuously evolving at very fast pace and are retraining models. As a result, in this work the responses generated by LLMs are indicative of the models' performance (i.e., GPT-4o for ChatGPT and Llama 3.1 for Meta AI) during specific time when our experiments were conducted, and hence it may not accurately represent their future behavior or capabilities.

Acknowledgments

We graciously acknowledge the support of the Prime Minister Research Fellowship (PMRF) Award and the SERB Core Research Grant (Grant Number: CRG/2022/005794) by the Government of India for carrying out this research.

References

- [1] [n. d.]. PromptBase. <https://promptbase.com/>. (Accessed on 26 September, 2024).
- [2] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [3] Yupeng Chang, Xu Wang, Jindong Wang, Yuan Wu, Linyi Yang, Kaijie Zhu, Hao Chen, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, et al. 2024. A survey on evaluation of large language models. *ACM Transactions on Intelligent Systems and Technology* 15, 3 (2024), 1–45.
- [4] Huashan Chen, Marcus Pendleton, Laurent Njilla, and Shouhuai Xu. 2020. A survey on ethereum systems security: Vulnerabilities, attacks, and defenses. *ACM Computing Surveys (CSUR)* 53, 3 (2020), 1–43.
- [5] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [6] Hanting Chu, Pengcheng Zhang, Hai Dong, Yan Xiao, Shunhui Ji, and Wenrui Li. 2023. A survey on smart contract vulnerabilities: Data sources, detection and repair. *Information and Software Technology* 159 (2023), 107221.
- [7] Josselin Feist, Gustavo Grieco, and Alex Groce. 2019. Slither: a static analysis framework for smart contracts. In *2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*. IEEE, 8–15.
- [8] João F. Ferreira, Pedro Cruz, Thomas Durieux, and Rui Abreu. 2021. SmartBugs: a framework to analyze solidity smart contracts. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (Virtual Event, Australia) (ASE '20)*. Association for Computing Machinery, New York, NY, USA, 1349–1352. <https://doi.org/10.1145/3324884.3415298>
- [9] Lianghong Guo, Yanlin Wang, Ensheng Shi, Wanjun Zhong, Hongyu Zhang, Jiachi Chen, Ruikai Zhang, Yuchi Ma, and Zibin Zheng. 2024. When to Stop? Towards Efficient Code Generation in LLMs with Excess Token Prevention. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1073–1085.
- [10] Hojae Han, Jaemin Kim, Jaeseok Yoo, Youngwon Lee, and Seungwon Hwang. 2024. ArchCode: Incorporating Software Requirements in Code Generation with Large Language Models. *arXiv preprint arXiv:2408.00994* (2024).
- [11] Rabimba Karanjai, Edward Li, Lei Xu, and Weidong Shi. 2023. Who is smarter? an empirical study of ai-based smart contract creation. In *2023 5th Conference on Blockchain Research & Applications for Innovative Networks and Services (BRAINS)*. IEEE, 1–8.
- [12] Heiko Koziol, Virendra Ashiwal, Soumyadip Bandyopadhyay, et al. 2024. Automated Control Logic Test Case Generation using Large Language Models. *arXiv preprint arXiv:2405.01874* (2024).
- [13] Moez Krichen, Meryem Ammi, Alaeddine Mihoub, and Mutiq Almutqi. 2022. Blockchain for modern applications: A survey. *Sensors* 22, 14 (2022), 5274.
- [14] Junjie Li, Aseem Sangal, Cheng Cheng, Yuan Tian, and Jinqiu Yang. 2024. Fine Tuning Large Language Model for Secure Code Generation. In *Proceedings of the 2024 IEEE/ACM First International Conference on AI Foundation Models and Software Engineering*. 86–90.
- [15] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2024. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *Advances in Neural Information Processing Systems* 36 (2024).
- [16] Zhijie Liu, Yutian Tang, Xiapu Luo, Yuming Zhou, and Liang Feng Zhang. 2024. No need to lift a finger anymore? assessing the quality of code generation by chatgpt. *IEEE Transactions on Software Engineering* (2024).
- [17] Loi Luu, Duc-Hiep Chu, Hrishi Olickel, Prateek Saxena, and Aquinas Hobor. 2016. Making smart contracts smarter. In *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*. 254–269.
- [18] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. 2024. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems* 36 (2024).
- [19] Shervin Minaee, Tomas Mikolov, Narjes Nikzad, Meysam Chenaghlu, Richard Socher, Xavier Amatriain, and Jianfeng Gao. 2024. Large language models: A survey. *arXiv preprint arXiv:2402.06196* (2024).
- [20] Mark Mossberg, Felipe Manzano, Eric Hennenfent, Alex Groce, Gustavo Grieco, Josselin Feist, Trent Brunson, and Artem Dinaburg. 2019. Manticore: A user-friendly symbolic execution framework for binaries and smart contracts. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1186–1189.
- [21] Consensus Mythx. 2018. Mythril. <https://github.com/Consensus/mythril>. (Accessed on 10 September, 2024).
- [22] OpenAI. 2024. OpenAI Cookbook. <https://github.com/openai/openai-cookbook>. (Accessed on 26 September, 2024).
- [23] Nii Osae Osae Dade, Margaret Lartey-Quaye, Emmanuel Teye-Kofi Odonkor, and Paul Ammah. 2023. Optimizing Large Language Models to Expedite the Development of Smart Contracts. *arXiv e-prints* (2023), arXiv–2310.
- [24] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2022. Asleep at the keyboard? assessing the security of github copilot's code contributions. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 754–768.
- [25] André Storhaug, Jingyue Li, and Tianyuan Hu. 2023. Efficient avoidance of vulnerabilities in auto-completed smart contract code using vulnerability-constrained decoding. In *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 683–693.
- [26] Sergei Tikhomirov, Ekaterina Voskresenskaya, Ivan Ivanitskiy, Ramil Takhaviev, Evgeny Marchenko, and Yaroslav Alexandrov. 2018. SmartCheck: Static Analysis of Ethereum Smart Contracts. In *2018 IEEE/ACM 1st International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*. 9–16.
- [27] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
- [28] Petar Tsankov, Andrei Dan, Dana Drachler-Cohen, Arthur Gervais, Florian Buenzli, and Martin Vechev. 2018. Securify: Practical security analysis of smart contracts. In *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*. 67–82.
- [29] Liyan Wang and Yves Lepage. 2022. Masked Prompt Learning for Formal Analogies beyond Words. In *IARML@IJCAI*. 1–14.
- [30] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.